

Templates (Genericos)

- una forma de escribir código que funciona con diferentes tipos de datos sin duplicar la lógica.
- se agrega a clases, interfaces y métodos con parámetros de tipo ($\langle T \rangle$, $\langle K, V \rangle$, etc). Luego estos parámetros son reemplazados por los tipos reales cuando se hace uso de la clase, interfaz o método.

Ejemplo

Clase template

```
public class Caja<T> {
    private T valor;

    public void guardar(T contenido) {
        this.contenido = contenido;
    }
    public T obtener() {
        return contenido;
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Caja<String> cajaDeTexto = new Caja<String>();
        cajaDeTexto.guardar("42");

        System.out.println("Caja de texto: " +
            cajaDeTexto.obtener());

        Caja<Integer> cajaDeEntero = new Caja<Integer>();
        cajaDeEntero.guardar(42);

        System.out.println("Caja de entero: " +
            cajaDeEntero.obtener());
    }
}
```

Ejemplo

Método template

```
public class Util {
    public static <T, U> void imprimir(T primero, U segundo) {
        System.out.println("Primero: " + primero + ", Segundo: "
            + segundo);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Util.imprimir("Hola", 123);
        Util.imprimir(3.14, true);
        Util.imprimir("Edad", 30);
    }
}
```

Ejemplo

Interfaz template

```
public interface Par<K, V> {
    K getClave();
    V getValor();
}
```

```
public class ParSimple<K, V> implements Par<K, V> {
    private K clave;
    private V valor;
    public ParSimple(K clave, V valor) {
        this.clave = clave;
        this.valor = valor;
    }
    @Override
    public K getClave() {
        return clave;
    }
    @Override
    public V getValor() {
        return valor;
    }
}
```


Type erasure

En Java los tipos genéricos se borran en tiempo de compilación.

En runtime sólo queda el tipo base (generalmente `Object`).

Esto significa que en memoria no se genera código para cada tipo utilizado en el template.

Cuando se obtiene un valor de un tipo genérico el compilador castea automáticamente al tipo deseado.

Ejercitación Registro

Implementar una clase registro que gestione elementos utilizando un `HashMap <K, V>`.

La clase deberá poder:

- Agregar elementos al `HashMap`
- Obtener el valor asociado a una clave
- Indicar si existe o no una clave en el `HashMap`.
- Listar todos los valores en el `HashMap`
- Listar todas las claves que estén asociadas a un valor en particular (se indica el valor por parámetro).

Ejercitación Vector

Refactorizar el código visto de TDA Vector para que haga uso de un genérico.